AeroTune Install & Run Guide

Updated for AeroTune V1.5.1

AeroTune is a local-first FPV Blackbox analysis tool for Betaflight tuning support. It reads flight-log data, runs parser diagnostics, analyzes roll/pitch/yaw behavior, compares before/after logs, and returns conservative tuning guidance instead of pretending to calculate perfect final PID values.

1. What AeroTune Supports

AeroTune supports these upload types:

```
.csv  - Betaflight Blackbox CSV export, analyzed directly
.bbl  - raw Blackbox log, converted locally with blackbox_decode first
.bfl  - raw Blackbox log, converted locally with blackbox_decode first
.txt  - raw Blackbox text log, converted locally with blackbox_decode first
```

CSV files work without extra tools. Raw `.bbl`, `.bfl`, and `.txt` uploads require Betaflight's `blackbox_decode` tool to be installed locally or pointed to with `BLACKBOX_DECODE_PATH`.

AeroTune does not bundle or redistribute `blackbox_decode`; it only calls a local copy when the user chooses to analyze raw Blackbox files.

2. Local Requirements

Required:

- Python 3
- pip
- Git
- AeroTune repository files

Optional for raw log upload:

- `blackbox_decode` from Betaflight blackbox-tools
- `make` and a compiler toolchain if building blackbox-tools locally

3. Clone and Install

```
git clone https://github.com/bostromdev/AeroTune.git
cd AeroTune
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
```

```
python3 -m pip install -r requirements.txt
python3 -m pip install python-multipart
```

4. Run AeroTune Locally

```
source .venv/bin/activate
python3 -m uvicorn main:app --reload
```

Open this in a browser:

```
http://127.0.0.1:8000
```

AeroTune is designed for local use because FPV Blackbox logs can be large. Local use avoids hosted upload limits, request timeouts, slow processing, failed large-file uploads, and accidental upload of private flight data to a server.

5. Optional Raw Blackbox Converter Setup

CSV export is the beginner-safe workflow. Use this section only when you want AeroTune to accept `.bbl`, `.bfl`, or `.txt` raw logs directly.

```
mkdir -p tools

if [ ! -d tools/blackbox-tools/.git ]; then
  git clone https://github.com/betaflight/blackbox-tools.git tools/blackbox-tools
else
  git -C tools/blackbox-tools pull --ff-only
fi

make -C tools/blackbox-tools obj/blackbox_decode

export BLACKBOX_DECODE_PATH="$PWD/tools/blackbox-tools/obj/blackbox_decode"
"$BLACKBOX_DECODE_PATH" --help
```

AeroTune searches for the converter in this order:

```
1. BLACKBOX_DECODE_PATH
2. tools/blackbox-tools/obj/blackbox_decode
3. system PATH
```

If raw upload fails, export CSV from Betaflight Blackbox Explorer and upload the CSV instead.

6. Recommended Blackbox Test Flight

For useful analysis, do not upload a random hover-only log. Use a repeatable 60-90 second test flight.

Recommended test pattern:

- Smooth cruise

- Small roll, pitch, and yaw inputs
- A few controlled throttle punches
- Medium turns
- Quick stops and direction changes
- One or two safe dirty-air / propwash recovery moments

Keep comparison flights as similar as possible:

- Same drone
- Same props
- Same battery type
- Same Blackbox settings
- Same tuning goal
- Same approximate flight length
- Same route or maneuver pattern

7. Workflow A: Single-Log Analysis

Use single-log analysis when you want AeroTune to answer:

```
What is wrong?
Which axis is affected?
Which PID/filter direction should I try?
Why is that recommendation being made?
```

Steps:

1. Export a Betaflight CSV, or upload a raw log if `blackbox_decode` is installed.
2. Choose drone size.
3. Choose tuning goal.
4. Upload the log.
5. Review parser report, converter report, axis cards, and tuning recommendations.
6. Make only one small change before testing again.

8. Workflow B: CSV Optimizer

Use the optimizer when a log has messy columns, repeated headers, metadata rows, or large exported structure.

AeroTune standardizes usable CSV data into this format:

```
time, gyro_x, gyro_y, gyro_z, setpoint_roll, setpoint_pitch, setpoint_yaw, throttle
```

The optimizer also returns parser diagnostics so users can understand missing columns, detected sample rate, detected duration, usable row count, repeated header cleanup, and optional missing fields.

9. Workflow C: V1.4 Before/After Comparison

Use comparison when you changed a tune and want to check if the after log improved.

Upload:

```
Before tune log
After tune log
```

AeroTune compares lower-is-better metrics such as:

- Tracking error ratio
- 95th-percentile absolute error
- Propwash-band energy
- High-frequency noise ratio
- Absolute tracking lag

The output gives an overall verdict, improvement score, confidence, axis-level results, strongest improvement axis, weakest axis, and next-step recommendation.

10. Workflow D: V1.5 Tune-Change Tracking

Use tune-change tracking when you want to validate a specific change.

```
1. Fly a clean before log.
2. Make one clear PID/filter/rate change.
3. Fly the same test again.
4. Upload before and after logs.
5. Write what changed.
6. Let AeroTune decide whether to keep, reduce, revert, or retest.
```

V1.5 checks:

- Overall before/after score
- Noise improvement
- Tracking improvement
- Propwash improvement
- Axis-specific verdicts
- Whether the after log is clean enough for style tuning
- Whether the change should be kept, reduced, reverted, or retested

AeroTune does not rewrite PID values automatically. The pilot still makes the final tuning decision.

11. Workflow E: V1.5.1 Saved Reports

When the tune-change form is submitted, AeroTune saves:

```
reports/tune_changes/<report_id>.json
reports/tune_changes/<report_id>.md
```

Saved reports include:

- Before filename
- After filename
- Tune-change notes
- Overall verdict
- Improvement score
- Confidence
- Noise improvement
- Tracking improvement
- Propwash improvement
- Axis verdicts
- Safe-baseline / style-tuning status
- Next step

Generated tune-change reports are local working files. They are ignored by Git by default so personal flight data is not accidentally committed.

12. Privacy and Local-Only Notes

AeroTune should be treated as a local analysis tool unless proper hosting, authentication, file retention, and security controls are added.

Do not publicly host the local app as-is with unrestricted uploads. Flight logs and generated reports can contain filenames, flight-analysis details, and user-written tuning notes.

13. Common Troubleshooting

The app will not start

Run from the repository root:

```
python3 -m uvicorn main:app --reload
```

Make sure dependencies are installed:

```
python3 -m pip install -r requirements.txt
python3 -m pip install python-multipart
```

Raw `.bbl`, `.bfl`, or `.txt` upload fails

Check the converter:

```
echo "$BLACKBOX_DECODE_PATH"
"$BLACKBOX_DECODE_PATH" --help
```

If the converter is missing, export CSV through Betaflight Blackbox Explorer and upload the CSV.

Parser report says required columns are missing

Use Betaflight Blackbox Explorer to export a full CSV from the correct flight segment. Avoid partial logs, unsupported CSV layouts, or files that only contain metadata.

Comparison confidence is low

Repeat the before/after tests with a more similar route, throttle pattern, battery type, props, and flight length.

14. License Reminder

AeroTune is source-available for personal non-commercial local evaluation. See `LICENSE` and `NOTICE` before copying, hosting, redistributing, modifying, forking, public release, commercializing, or reusing AeroTune code, logic, UI, docs, branding, or project materials.

Last Page: Converter and CSV Explanation

Why AeroTune Uses CSV for Analysis

AeroTune analyzes CSV because CSV is the clean, table-shaped version of a Blackbox flight log. Once the data is in CSV form, the analyzer can reliably read columns like time, gyro, setpoint, throttle, and axis movement.

Raw `.bbl`, `.bfl`, and `.txt` Blackbox files are not the best direct analysis format because they are recorder/export formats. They can include binary structure, multiple flights, metadata, firmware-specific encoding, and details that need to be decoded before normal data analysis can happen.

How Raw Blackbox Conversion Works

When a raw Blackbox file is uploaded and `blackbox_decode` is available, AeroTune uses this flow:

```
Raw .BBL / .BFL / .TXT
        ↓
blackbox_decode converts the raw log into CSV
        ↓
AeroTune parser reads the CSV columns
        ↓
AeroTune analyzer checks tracking, noise, propwash, lag, and axis behavior
        ↓
AeroTune gives conservative tuning guidance
```

In other words, AeroTune can accept raw logs, but it still analyzes the decoded CSV version. That is intentional. The converter handles the Betaflight log format; AeroTune handles the tuning logic.

Beginner Recommendation

Beginners should start with Betaflight Blackbox Explorer and export CSV manually. It is easier to inspect, easier to name correctly, easier to verify, and easier to troubleshoot.

Advanced users can install `blackbox_decode` so AeroTune can convert raw logs locally.

Why AeroTune Does Not Bundle blackbox_decode

AeroTune keeps `blackbox_decode` external because it belongs to the Betaflight / blackbox-tools ecosystem. AeroTune calls a local copy only when the user installs it. This keeps the project cleaner, avoids bundling a separate converter binary, and makes it clear that AeroTune's job is analysis after decoding.